

Introduzione al C

Lezione 5

Sistemi Operativi - II Modulo (Canale A - L)

Prof. Paolo Zuliani

Dipartimento di Informatica

(Materiale didattico per cortesia del Prof. Emiliano Casalicchio)



SAPIENZA
UNIVERSITÀ DI ROMA



Agenda

- Ambiente di sviluppo ed esecuzione
- Come compilare ed eseguire un programma C
- Output/input
- Variabili ed espressioni

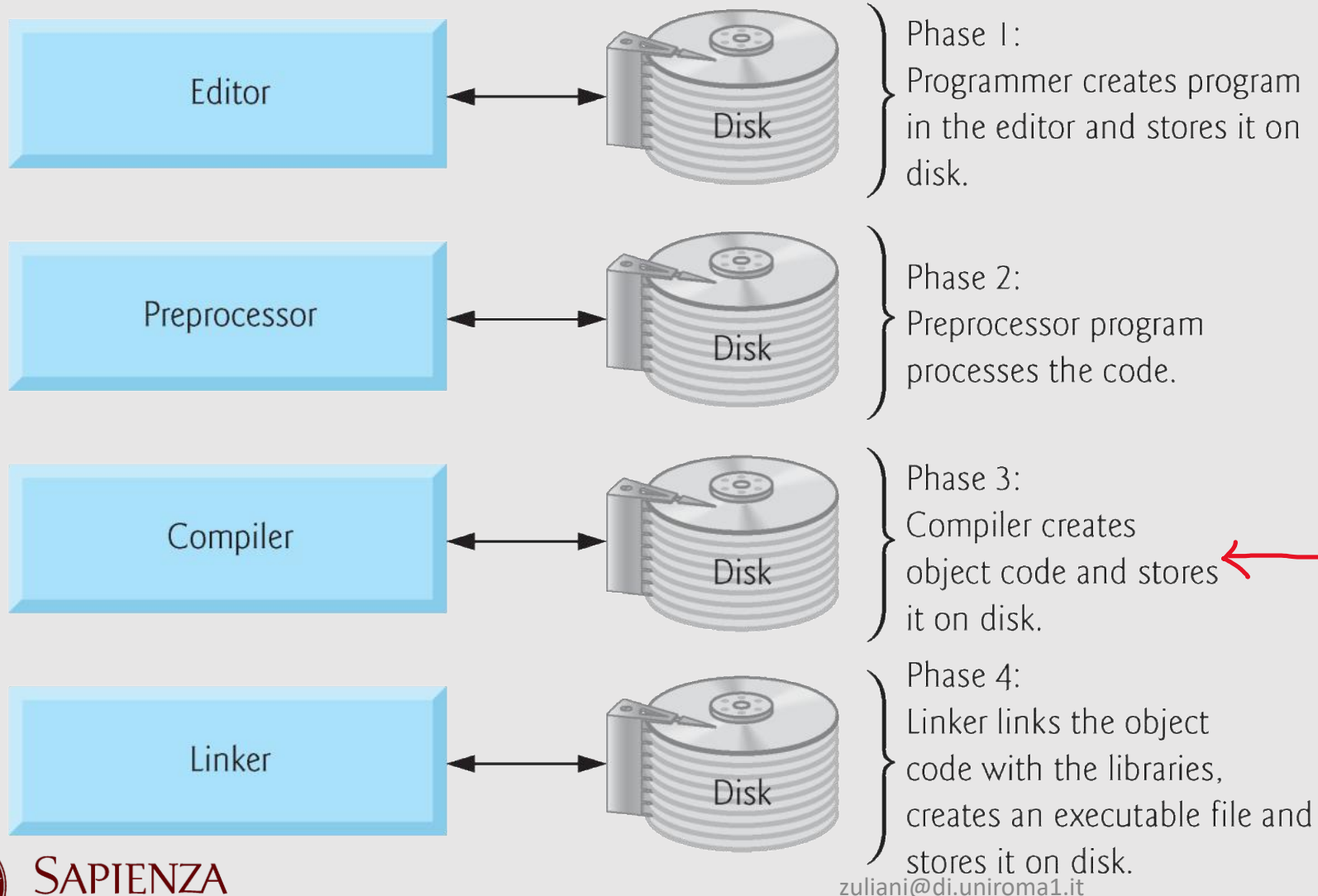


Linguaggio di Programmazione C

- Sviluppato da AT&T Bell Labs agli inizi degli anni '70
- Usato per sviluppare il 'nucleo' di Unix (e successivamente Linux e altri sistemi operativi)
- Standardizzato da *American National Standards Institute* (ANSI) e poi anche da *International Organization for Standardization* (ISO)
- E' *indipendente* dall'hardware (come Java, ad es.), quindi le applicazioni scritte in C possono essere eseguite (senza alcuna o con poche modifiche) su ogni tipo di processore (portabilità)
 - Previa eventuale ri-compilazione



Ambiente di sviluppo in C



- gcc (GNU Compiler Collection) include il compilatore C che svolge le attività di
 - Pre-processamento, compilazione, linking

Produce codice oggetto, memorizzato in un file .o

Differenze con Java (e Python)

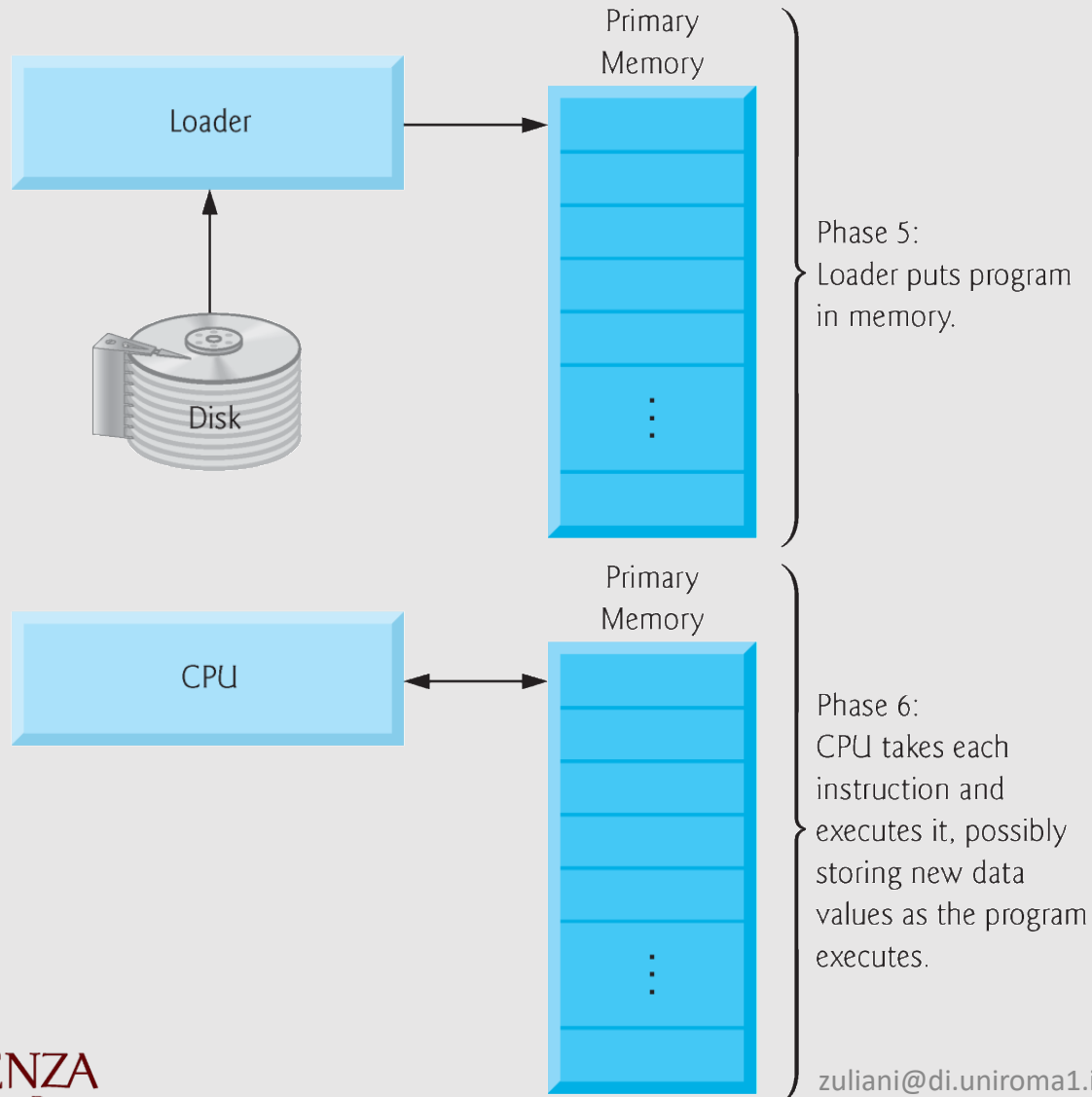
- La compilazione di un programma in Java mediante javac non produce codice eseguibile ma codice interpretabile dalla Java Virtual Machine (JVM) contenuto nel .class (bytecode)
- Quando voglio eseguire un .class lo dò in pasto alla JVM
 - Il vero processo in esecuzione è la JVM che esegue il .class interpretandolo
- La “compilazione” di un programma in C mediante gcc (fasi 2 - 4) produce un “object code” che è un file eseguibile
 - Eseguendo il file viene creato un processo indipendente dal gcc che eventualmente posso eseguire anche su altri sistemi senza necessità di ri-compilarlo.
 - Lo stesso è valido per un programma scritto in C++
 - Al contrario, il file .class dovrò sempre darlo in pasto ad una JVM
 - [GraalVM permette di compilare codice Java producendo un eseguibile]

File eseguibile in Linux

- Risultato di una compilazione mediante gcc (o di altro compilatore) di un programma C, C++, assembly
- Script con lo *shebang*
 - *(vedremo più in là il significato)*



Ambiente di esecuzione in C

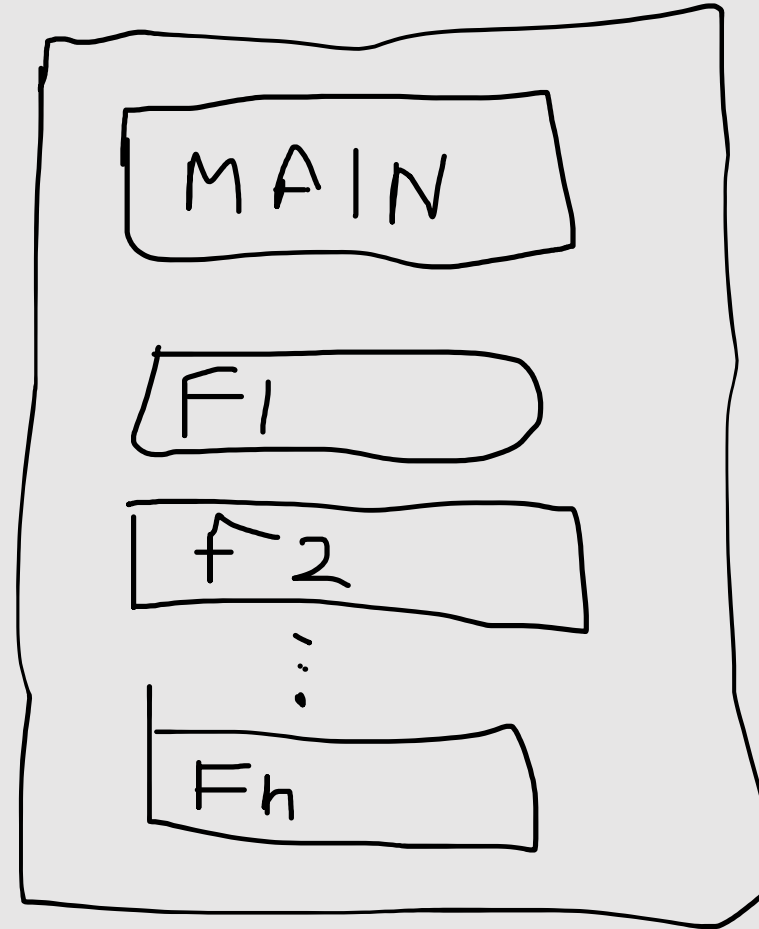


Il C

- Linguaggio di alto livello (anche detto «assembly portabile»), un programma è una sequenza di istruzioni
 - `int x=0; y[1]=3` ←
 - `if a==b y=y+1`
 - ...
- Le fasi e i tool per creare ed eseguire un programma C sono:
 - Edit (`vi`, `gEdit`, `pico`, `emacs`, `nano`, ...)
 - Pre-process (`cpp`, ma invocato direttamente da `gcc`)
 - Compile (`gcc`)
 - Link (`ld`, invocato direttamente da `gcc`)
 - Load (SO)
 - Execute (SO)

Struttura di un programma C

- `main()` function (**OBBLIGATORIA**)
 - Può semplicemente essere il punto da cui vengono invocate tutte le funzioni che compongono il programma o può contenere una logica complessa
 - Può anche essere l'unico blocco di codice del programma
- Function i (F_i)
 - Un blocco di codice che esegue uno specifico compito
 - Identificato da un nome univoco
- `main()` e le altre funzioni possono risiedere nello stesso file `.c` o in file diversi `.c`



Funzioni

- Ogni funzione consiste di una intestazione (header) e di un blocco

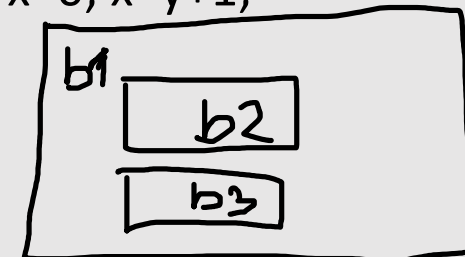
```
<return-type> function-name (parameter-list)  
    basic block
```

header

- Basic block

```
{  
    declaration of variables  
    executable statements  
}
```

- ; usato per terminare uno *statement* (dichiarazione): `int x,y; x=0; x=y+1;`
- Un blocco può contenere 0 o più statement
- I blocchi possono essere annidati



Return statement

`return expression`

- imposta il valore di ritorno al valore di `expression`
- Ritorna il valore alla funzione chiamante

```
int main()           // header
{                   // beginning of basic block
    // ...
    return 0;        // program ending successfully
}                   // end of basic block
```

- `expression` può essere una costante, una variabile, un'espressione logico-aritmetica, una funzione; ad es:
 - `return 1;`
 - `return x;`
 - `return 3*x+1;`
 - `return fact(x);`

Primo programma

```
#include <stdio.h>
```

```
int main() {
```

```
    printf("Il mio primo programma C: Hello World! \n");
```

```
    return 0; /*indica che il programma ha terminato con  
             successo*/
```

```
}
```

Potete scrivere questo programma e salvarlo in un file `helloworld.c`

Per compilare ed eseguire

```
gcc -Wall prog-name.c
```

- In questo modo vengono stampati tutti i messaggi di avvertimento/warning (se ci sono!!!)

```
gcc -Wall prog-name.c -lm
```

- `-lm` va specificato se si includono le librerie matematiche `<math.h>`, ad esempio per usare funzioni come `sin`, `cos`, `log`, `ln`, ...

- Il risultato è un file eseguibile `a.out` che potete eseguire digitando `./a.out` al prompt della shell
- Per specificare il nome dell'eseguibile in output usare l'opzione `-o`

```
gcc -Wall prog-name.c -o executable-name.o
```



Precompilazione, compilazione e linking

- Solo precompilazione/preprocessamento
 - `cpp helloworld.c > precompilato.c`
 - Esegue tutte le direttive del compilatore; elimina i commenti
 - vediamo cosa c'è in `precompilato.c`
- Solo compilazione (di un precompilato)
 - `gcc -c precompilato.c -o covid19.o`
 - controlla che la sintassi sia a posto
 - per ogni chiamata a funzione, controlla che venga rispettato la corrispettiva intestazione (che quindi deve esistere al momento della chiamata!)
 - crea effettivamente del codice macchina, ma solo per il contenuto delle funzioni
 - ogni chiamata a funzione ha una destinazione simbolica

Precompilazione, compilazione e linking (cont.)

- Solo precompilazione+compilazione
 - `gcc -c file.c -o file.o`
- Solo linking
 - `gcc file.o`
 - risolve tutte le chiamate a funzione: adesso, per ogni funzione chiamata non basta più l'intestazione, ci deve essere anche l'implementazione (blocco di istruzioni)
 - l'implementazione può essere o data dal programmatore (vedere la funzione nell'esempio fornito) o fornita da librerie di sistema (come per la funzione `printf`)
 - L'inclusione delle librerie può essere automatica o specificata dall'utente
 - Ad esempio la libreria `libc.a` che contiene la `printf` è inclusa automaticamente

Precompilazione, compilazione e linking (cont.)

- Linking di più file
 - `gcc file1.o file2.o file3.o`
 - si possono anche mischiare file `.c` (sorgenti) e file `.o` (oggetti)
- tutto (un solo file): `gcc file.c`
- tutto (piu file): `gcc file1.c ... filen.c`



Commenti

- Linee di testo ignorate dal compilatore
- Due modalità: `//` e `/* */`

```
#include <stdio.h>

int main() {
    printf("Hello World!!!!\n"); //stampa a video
    return 0; /*indica che il programma ha terminato
               con
               successo*/
}
```

Direttive al preprocessore

- Con # si indicano le direttive a preprocessore
- `#include filename` dice di includere il contenuto di `filename` specificato al posto di #
- Il file (usualmente `.h`) è detto *header* file
- `#include <stdio.h>`
 - `<>` indicano che il file header è un file standard del C in `/usr/include`
 - `""` indicano che il file header è dell'utente e si trova nella directory corrente o in un path specificato
 - `-I` permette di specificare le directory in cui cercare gli header file

Esercizio 1

- Compiliamo ed eseguiamo `hello_world` (lo facciamo insieme)
- Provare a fare in modo che `hello_world.1.c` sia incluso direttamente dentro `hello_world.2.c`. Si può ancora fare la compilazione separata (ovvero, prima si compilano separatamente `hello_world.1.c` e `hello_world.2.c`, e poi si linkano insieme)? Si può fare anche l'inclusione inversa?
- Il codice si trova nel file archivio `lezione5.tgz`
 - Per scompattare il file usate: `tar -xzf lezione5.tgz`



Perché devo scrivere ./a.out?????

- La shell cerca i comandi in alcune cartelle specificate all'avvio della shell stessa
- Digitate il comando `env`
- Stampa una lista di “variabili di ambiente” (environment variables)
- Cercate la variabile **PATH**: contiene una lista di cartelle dove la shell cerca i file eseguibili
- Digitando `./a.out` “diciamo” alla shell che `a.out` e' nella `cwd`!!

Input e Output

- Input: letto da tastiera o altro dispositivo e memorizzato in variabili
- Output: visualizzato a monitor o inviato a altro dispositivo (es. stampante) prendendo valori da variabili
- L'ambiente run-time del C, quando un programma viene eseguito, apre tre file: `stdin`, `stdout` e `stderr`
- Tutte le funzioni essenziali per l'I/O sono nell'include file `stdio.h`

Output

- `printf("format string", value-list);`
- `value-list` può contenere
 - sequenze di caratteri
 - variabili
 - costanti
 - espressioni logico-matematiche
- `printf` riceve valori, ma il C permette di manipolare anche indirizzi di memoria e passarli come input a funzioni.
 - Per stampare il contenuto di una locazione di memoria di cui si conosce l'indirizzo si usa `scanf`
- Vedremo che ci sono altre funzioni per gestire l'output

Output

- Nella printf possiamo controllare la spaziatura orizzontale e verticale, e l'output di caratteri speciali usando le sequenze di *escape* \
- `printf("#iorestoacasa
#andratuttobene !!!!\n");`
**//dopo aver stampato la stringa
va a capo**

\a

Alarm or Beep

\b

Backspace

\f

Form Feed

\n

New Line

\r

Carriage Return

\t

Tab (Horizontal)

\v

Vertical Tab

\\

Backslash

\'

Single Quote

\"

Double Quote

\?

Question Mark

\ooo

octal number

\xhh

hexadecimal number

\0

Null

<https://www.geeksforgeeks.org/escape-sequences-c/>



Esercizio 2

- Spiegare l'output del seguente codice:

```
#include <stdio.h>
int main()
{
    printf("aaaaa\nbbbbbb\f\rccccc\r\fddddd\reeeee\fffffff\n");
}
```


Variabili ed espressioni



Parole riservate e identificatori

- In C esistono parole riservate, ovvero che appartengono all'«alfabeto» del linguaggio
 - `if`, `return`, `do`, `for`, `while`, `int`, `double`, ...
- Un *identificatore* è una parola che viene utilizzata per identificare una variabile o costante



VALIDI

`distance`
`distance32`
`milesPerHour`
`_voltage`
`goodChoice`
`MIN_RATE`
`Min_rate`
`min_rate`

NON VALIDI

`x-ray`
`2ndGrade`
`$amount`
`two&four`
`after five`
`return`



Regole per costruire identificatori validi

1. Il primo carattere in un identificatore deve essere una lettera o underscore (_) e può essere seguito solo da qualsiasi lettera, numero o underscore
2. Non devono iniziare con una cifra (perché?)
3. Le lettere MAIUSCOLE e minuscole sono *distinte*. Cioè, gli identificatori fanno distinzione tra maiuscole e minuscole
4. Le virgole o gli spazi vuoti non sono consentiti all'interno di un identificatore
5. Le parole chiave non possono essere utilizzate come identificatore
6. Gli identificatori non devono avere una lunghezza superiore a 31 caratteri



Esercizio (da fare tra qualche giorno)

- Scrivere un programma che controlli se un identificatore è valido ovvero, data una stringa verificare che siano rispettate le condizioni 1 - 6 elencate nella slide precedente.
- *NOTA: ora non avete tutte le conoscenze per risolvere il problema ... ma se qualcuno le ha può iniziare a cimentarsi.*

Variabili

- Una locazione di memoria dove può essere memorizzato un valore che verrà utilizzato da un programma
- I valori nelle variabili possono essere modificati una o più volte durante l'esecuzione di un programma
- Una variabile può contenere un solo valore alla volta, ovvero se scrivo in una variabile, il valore attualmente memorizzato viene sovrascritto

```
int x=1;
printf(``Questo è il valore di x %d \n'', x);
x=3;
printf(``Questo è il nuovo valore di x %d \n'', x);
```

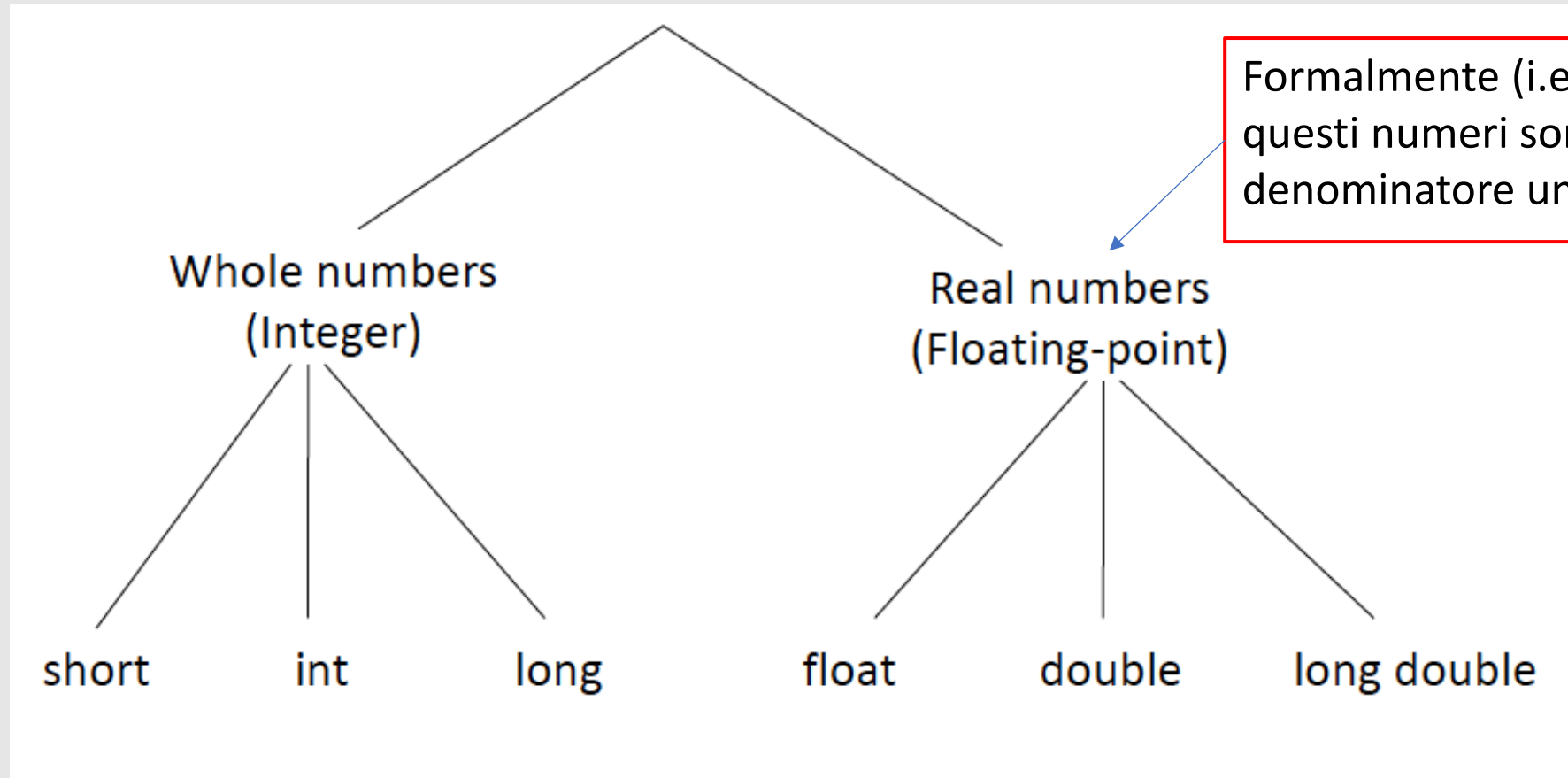
Variabili (cont.)

- Deve essere un identificatore valido
- Va dichiarato il nome e tipo *prima* di poterla utilizzare
`int x; double y; char c;`
- Il nome dovrebbe essere descrittivo del contenuto

Dichiarazione di variabili e costanti

- `optional_modifier data_type name_list;`
- `optional_modifier`
 - signed, unsigned
 - short, long
 - const
- `data_type`
 - specifica il tipo di valore
 - permette al compilatore di sapere quali sono le operazioni consentite sul tipo di dato e come deve essere rappresentato in memoria
- `name_list`
 - lista di identificatori

Tipi di dato numerici



Tipo	Dimensione	Intervallo di valori
char	1 byte	-128 a 127 o 0 a 255
unsigned char	1 byte	0 a 255
signed char	1 byte	-128 a 127
int	2 or 4 bytes	-32,768 a 32,767 or -2,147,483,648 a 2,147,483,647
unsigned int	2 or 4 bytes	0 a 65,535 or 0 a 4,294,967,295
short	2 bytes	-32,768 a 32,767
unsigned short	2 bytes	0 a 65,535
long	8 bytes	-9223372036854775808 a 9223372036854775807
unsigned long	8 bytes	0 a 18446744073709551615

Tipo	Dimensione	Intervallo di valori	Precisione
float	4 byte	1.2E-38 a 3.4E+38	6 decimal places
double	8 byte	2.3E-308 a 1.7E+308	15 decimal places
long double	10 byte	3.4E-4932 to 1.1E+4932	19 decimal places

Ci sono costanti che contengono le dimensioni dei vari tipi numerici

Esercizio 3

- Guardare il codice in `integerSize.c` e `realSize.c`
- Compilarlo ed eseguirlo
- Cercare di capire perché serve includere le librerie `stdlib`, `limits` e `float` (sono in `/usr/include`)
 - `#include <stdlib.h>`
 - `#include <limits.h>`
 - `#include <float.h>`
- La dimensione di un tipo di dato può essere ottenuta anche usando la funzione `sizeof(datatype)`
- Riscrivere i 2 programmi usando `sizeof`



Caratteri char (1byte, tutti i caratteri ASCII)

<http://www.asciitable.com/>

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040	 	Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	\$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174	|	
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177		DEL

Source: www.LookupTables.com



Boolean data type

- `_Bool`
 - Può memorizzare solo 0 e 1
 - Valori diversi da 0 vengono memorizzati come 1
- `bool`
 - Richiede l'uso di `<stdbool.h>`
 - Memorizza `true` e `false`
- In ogni espressione logica
 - Risultato 0 significa `false`
 - Risultato 1 significa `true`

Dichiarazione di variabili (già vista!!)

```
int age;

short first_reading;
short int last_reading;

long first_ssn;
long int last_ssn;

float interest_rate;
double division_sales;

char grade, midInitial;
```

Assegnazione di valori a variabili

- Quando una variabile viene dichiarata e non inizializzata assume un valore *indeterminato*
 - Perché? Provate
- Assegnazione di un valore può essere fatta
 - In fase di dichiarazione
 - `int x=3, y=2; char c='a'; double dd=9.3;`
 - In fase di elaborazione mediante l'operatore di assegnazione =
 - Leggendo un valore in input, ad esempio con `scanf`

Ancora su assegnazione

- = operatore binario
- `l_value = r_value`
 - `l_value` solitamente una variabile o un puntatore
 - `r_value` un valore o una espressione
- E' possibile scrivere cose tipo
 - `a = b = 0;`
 - In fase di esecuzione prima viene valutata l'espressione `b=0` e poi il risultato, ovvero il valore di `b` viene assegnato ad `a`.
- Un esempio di espressione
 - `x = y + 5;`
 - Anche qui' viene prima valutata l'espressione e viene prodotto il risultato (`r_value`) che viene assegnato a `x` (`l_value`)

Altri esempi di assegnazione

```
distanza = velocita * tempo;
```

```
dailyRate= 65.3;
```

```
hours=8;
```

```
hourlyRate = pay/hours;
```

E se hours=0? Provate!!

Output di variabili

- Vediamo come con `printf` possiamo scrivere su `stdout` il valore di una variabile (lo avete già visto nell'esempio precedente!!!)
 - `printf(format_string, expression_list)`
 - `format_string` deve contenere dei *placeholders* o «segnaposti»
 - Un segnaposto inizia con %
 - Un segnaposto dice che al suo posto ci andrà il valore di una variabile e che tipo di dato deve essere scritto
 - Ad es:

- `printf("%d, %l", integerValue, longVariable);`

Stampa un intero

Stampa un intero «lungo»

Stampa una virgola e uno spazio



Segnaposti più comuni per integer e float

- %d oppure %i per interi, %l per long
- %o per interi in ottale
- %x per interi in esadecimale
- %f, %e, %g per virgola mobile
 - f - formato standard
 - e - notazione scientifica
 - g - sceglie automaticamente il formato migliore tra f ed e
- %lf per double

```
printf("x = %d\n", x);  
printf("%d + %d = %d\n", x, y, x+y);  
printf("Rate is %d%%\n", rate*100);
```

E' possibile comporre stringhe che vengono completate dal valore delle variabili

Ancora su printf

- La funzione restituisce come risultato il *numero di caratteri stampati*

```
int num1 = 55;  
int num2 = 30;  
int sum = num1 + num2;  
int printCount;  
printCount = printf("%d + %d = %d\n", num1, num2, sum);
```

- **Esercizio 4:** Che valore assumerà `printCount`? Provate!! Anche con diverse stringhe e numero di variabili

Il formato completo di un segnaposto

- `% [parameter] [flags] [width] [.precision] [length] type`
- Al link https://en.wikipedia.org/wiki/Printf_format_string andando nella sezione “Format placeholder specification” trovate tutti i dettagli
- **Esercizio 5:** studiate la pagina wikipedia e scrivete un programma C in cui mostrate varie opzioni di formattazione per i tipi di dato reali, interi e carattere usando tutti i campi sopra indicati, ovvero
`[parameter] [flags] [width] [.precision] [length]`

Costanti

- Ovvero i valori che assegnate alle variabili
- **Interi:**
 - 23, 59874, -89, etc.
 - Non vanno usati delimitatori (ad es. per le migliaia)
 - In modalità decimale come sopra oppure in
 - *Ottale* 132_8
 - *Esadecimale* $0x5A$
 - *Carattere ASCII* 'Z'
 - I tre esempi sopra corrispondono al decimale 90
 - $90 = 132_8 = 0x5A = \text{'Z'}$



Costanti

- Il C è molto 'liberale': le seguenti istruzioni sono valide (!)

```
int x;
```

```
x = 47 + 'Z' + 0x5A;
```

Quanto vale x?

227

Costanti

- **Virgola mobile** (*floating point*):
 - Decimale: 3.14, 2.0, ...
 - Notazione scientifica: 5.5E10 ($= 5.5 \times 10^{10}$)
- Le seguenti istruzioni sono valide:

```
float f;
```

```
f = 47.1 + 'Z' + 0x5A;
```

Quanto vale `f`?

227.100006103515625

0.1 non può essere scritto come una frazione con denominatore una potenza di 2!



Costanti

- **Caratteri:**

- 'A', 'a', '€', '{', etc.
- Segnaposto: %C

- **Stringhe:**

- "La Sapienza", "Questa è una stringa", "1", etc.
- Segnaposto: %S



Input da terminale - scanf

- `scanf(format-string, address-list)`
- `format-string`
 - Contiene segnaposti che dicono a scanf in che tipo di dato la stringa in input viene convertita
 - https://en.wikipedia.org/wiki/Scanf_format_string
- `address-list`
 - contiene gli indirizzi di memoria in cui devono essere memorizzati i valori ricevuti in input (in effetti, una lista di *puntatori*)
- Es: `int peso; scanf("%d", &peso);`
- `peso` è una variabile intera; l'operatore `&` estrae il suo indirizzo di memoria e lo passa a `scanf`

Ancora su scanf

- La funzione restituisce come risultato il *numero di valori di input letti*

```
int dataCount;  
dataCount = scanf("%d %d", &height, &weight);
```

- **Esercizio 6:** Che valore assumerà dataCount? Provate!! Anche con diverso numero e tipo di variabili

Esercizio 7: scanf

- Basandosi su questo esempio scrivete un programma che consente di leggere diversi tipi di dati in input da tastiera
- Provate anche a leggere più dati con la stessa scanf
- Cosa accade se inserite un dato di input sbagliato ovvero di un formato diverso da quello atteso? Ad esempio un carattere al posto di un intero?

```
int main() {  
    // declare variables  
    int x;  
    int y;  
    int sum;  
  
    // read values for x and y from standard input  
    printf("Enter value for x: ");  
    scanf("%d", &x);  
  
    printf("Enter value for y: ");  
    scanf("%d", &y);  
  
    sum = x + y;  
  
    // print  
    printf("x = %d\n", x);  
    printf("y = %d\n", y);  
    printf("x + y = %d\n", sum);  
  
    printf("%d + %d = %d\n", x, y, sum);  
    printf("%d - %d = %d\n", x, y, (x - y));  
    printf("%d * %d = %d\n", x, y, (x * y));  
    return 0;  
}
```



Operatori

- Aritmetici: $+$, $-$, $*$, $/$, $\%$
- Relazionali: $==$, $!=$, $<$, $<=$, $>$, $>=$
- Logici: $!$, $\&\&$, $||$
- Bit a bit (bitwise): $\&$, $|$, $\sim$, $\wedge$
- Shift (scorrimento): $<<$, $>>$

Operatori aritmetici - regole di precedenza

- $()$ valutate per prime. In caso di parentesi annidate si inizia a valutare dalla più interna
 - $x = a + (a + (3/5 + c * (c - 3)))$
- $*$, $/$, o $\%$ valutate per seconde. Se ce ne sono diverse la valutazione è da sinistra a destra
 - $x = a * b / c$
- $+$ o $-$ valutate per ultime
 - $x = a + b + c / 3 ???$
 - Come devo scrivere per calcolare la media delle variabili a, b e c?

Operatore di divisione /

- Il risultato dipende dai tipi di dato degli operandi
- Il risultato è dello stesso tipo di dato dell'operando più grande (in termini di tipo di dato)
 - Vediamo il file `divisione.c`
- Esercizio
 - Verificare in che ordine vengono fatte le divisioni delle seguenti espressioni e calcolatene il risultato
 - $11 / 2 / 2.0 / 2$
 - $11 / 2 / 2 / 2$

Operatore resto %

- Ritorna il resto della divisione
 - $7\%2=1$ $(7 = 2 \times 3 + 1)$
 - $7\%5=2$ $(7 = 1 \times 5 + 2)$
 - ...

Altre regole

- Troncamento

```
int x;
```

```
x=3.14 // x conterrà solo 3, senza avvertire
```

- *Casting* (conversione); vedi file `casting.c`

Abbreviazioni

Assegnamento	Abbreviazione (shortcut)
<code>d = d - 4</code>	<code>d -= 4</code>
<code>a = a + 3</code>	<code>a += 3</code>
<code>e = e * 5</code>	<code>e *= 5</code>
<code>f = f / 3</code>	<code>f /= 3</code>
<code>g = g % 9</code>	<code>g %= 9</code>

Abbreviazioni

- `intVar++` corrisponde a `intVar=intVar+1`
- `intVar--` corrisponde a `intVar=intVar-1`
- Due opzioni:
 - Post-incremento `x++` (prima usa il valore di *x*, *poi* incrementa *x*, cioè *x=x+1*)
 - Pre-incremento `++x` (prima incrementa *x*, cioè *x=x+1*, *poi* usa il valore di *x*)
 - Vedi file `prepost.c`

Domande?

